MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

Verification Using SystemVerilog Internal Assessment

Submitted by

Registration Number	Roll No.	Name
240959008	5	Shivansh Bose
240959024	10	Garv Kishore Gupta
240959266	56	Paarth Sud
240959304	64	Priyangshu Agarwala

SCHOOL OF ELECTRICAL AND ELECTRONICS ENGINEERING

MANIPAL-576104, KARNATAKA, INDIA

OCTOBER 2026

ABSTRACT

SystemVerilog supports both hardware description and verification, making it suitable for modelling digital circuits and checking their behavior. This report presents a class-based verification environment for a JK flip-flop, a sequential logic element whose output depends on its present state and the J and K inputs. The objective is to implement the flip-flop and verify its expected behavior at the active clock edge.

The design is written as synthesizable-style RTL with an active-high synchronous reset. A SystemVerilog interface groups the DUT signals. A transaction object carries input and output values, while a generator creates an initial reset transaction followed by randomized input combinations. A driver applies the transactions, a monitor samples the response, and a scoreboard calculates the expected output for comparison.

The supplied waveform shows the clock, J, K, Q, and reset signals over time. It illustrates reset activity and changes in the input controls alongside the output response. The verification environment is structured to report PASS when the observed output equals the scoreboard's expected value and FAIL otherwise. The waveform is included as the visual result record provided for this report.

The work demonstrates how a small sequential circuit can be tested using a reusable, class-based verification flow. The result is a modular testbench with separate stimulus generation, driving, observation, and checking components. The design and testbench code are provided in the annexure. Tools/resources: SystemVerilog, EDA Playground, and the supplied waveform capture.

CONTENT

Abstract

Chapter 1 INTRODUCTION

1.1 Introduction

1.2 Objectives

Chapter 2 METHODOLOGY

2.1 Design Methodology

2.2 Verification Environment

Chapter 3 RESULT ANALYSIS

3.1 Output Waveform

3.2 Signal and Verification Analysis

Chapter 4 CONCLUSION

SYSTEMVERILOG CODES

CHAPTER 1
INTRODUCTION

1.1 Introduction

This chapter introduces the JK flip-flop and the purpose of verifying its sequential behavior. A JK flip-flop is a clocked storage element with two control inputs, J and K, and one output, Q. Its behavior is determined at the rising edge of the clock, subject to the synchronous reset.

The report follows the supplied institutional report format: it describes the design and verification methodology, discusses the provided output waveform, and concludes with possible extensions. The SystemVerilog implementation and verification classes are included in the annexure.

1.2 Objectives

- Implement a JK flip-flop using SystemVerilog RTL.
- Apply an active-high synchronous reset and verify hold, reset, set, and toggle behavior.
- Create a class-based testbench with generator, driver, monitor, and scoreboard components.
- Observe the DUT response and compare it with an independently maintained expected value.

CHAPTER 2
METHODOLOGY

2.1 Design Methodology

The DUT is implemented in module jk_ff. On each positive edge of clk, rst is checked first. When rst is high, Q is cleared to 0. Otherwise, the concatenated input {J,K} selects one of four operations.

J	K	Operation at rising edge
0	0	Hold current Q
0	1	Clear Q to 0
1	0	Set Q to 1
1	1	Toggle Q

The reset is synchronous: it affects Q only when a rising clock edge occurs while rst is asserted. The clock in the testbench starts at 0 and toggles every 10 simulation time units.

2.2 Verification Environment

The verification environment is divided into small components. The generator sends one reset transaction followed by twelve randomized transactions through mailbox G2D. The driver receives each transaction and applies rst, J, and K on the falling clock edge. The monitor samples the inputs at the rising edge, waits briefly for nonblocking assignments to update, captures Q, and sends the observation through mailbox M2S. The scoreboard updates expected_Q according to the JK truth table and reports PASS or FAIL.

The transaction class contains randomized J, K, and rst fields, an observed Q field, a display function, and a copy function. The interface jk_ff1 groups the signals and is passed to the driver and monitor using a virtual interface.

CHAPTER 3
RESULT ANALYSIS

3.1 Output Waveform

Figure 3.1 shows the supplied waveform capture. The displayed signals are clk, J, K, Q, and rst; the signal set appears twice in the screenshot. The red-highlighted region at the beginning indicates the initial Q display/selection region in the waveform viewer.

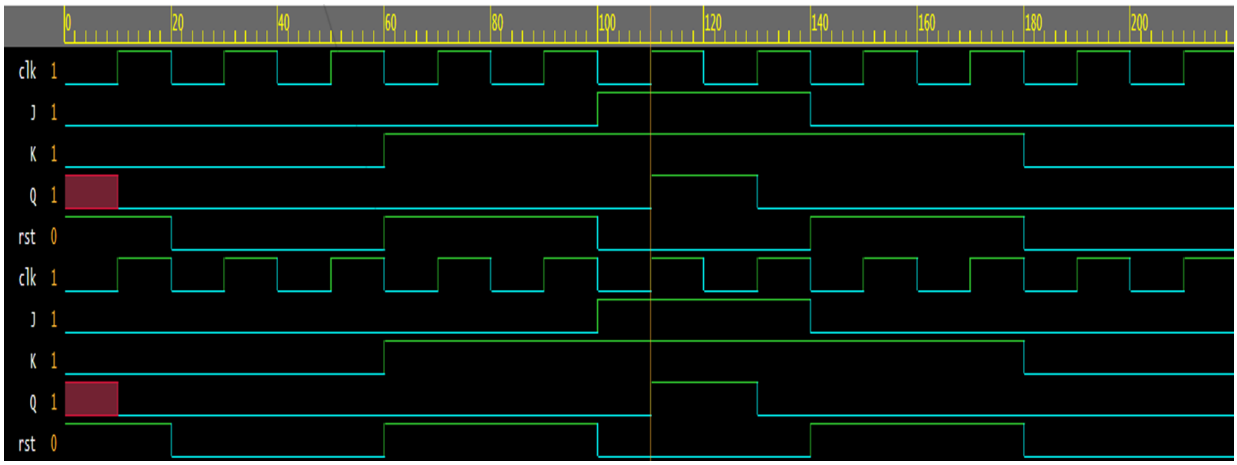


Figure 3.1: Supplied JK flip-flop output waveform.

3.2 Signal and Verification Analysis

The clock provides the timing reference for the sequential circuit. J and K change during the simulation, while rst alternates between asserted and deasserted intervals. Since the reset is synchronous, Q is expected to clear at a rising edge when rst is high. When reset is low, Q follows the selected JK operation at the rising edge.

The waveform is included as supplied and is not presented as a substitute for a full simulator transcript. The scoreboard code is intended to compare each sampled Q value with expected_Q

and print a PASS or FAIL message. The screenshot alone does not show the simulator console, so the report does not claim a specific count of successful checks.

CHAPTER 4
CONCLUSION

4.1 Work Conclusion

A SystemVerilog implementation and class-based verification structure for a JK flip-flop have been documented. The design includes an active-high synchronous reset and implements hold, reset, set, and toggle operations. The testbench separates stimulus generation, signal driving, monitoring, and expected-value checking, supporting a modular verification approach.

The supplied waveform provides a visual view of the clock, reset, input controls, and output. The scoreboard provides the mechanism for checking observed output values against expected behavior.

ANNEXURE A | COMMENTED SYSTEMVERILOG CODE

A.1 RTL - jk_ff.sv

```
module jk_ff (
    input logic J, K, clk, rst,
    output logic Q
);
    // Update the output only on a rising clock edge.
    always_ff @(posedge clk) begin
        // Active-high synchronous reset has priority.
        if (rst)
            Q <= 1'b0;
        else begin
            case ({J, K})
                2'b00: Q <= Q;   // Hold
                2'b01: Q <= 1'b0; // Reset
                2'b10: Q <= 1'b1; // Set
                2'b11: Q <= ~Q;   // Toggle
                default: Q <= 1'b0;
            endcase
        end
    end
endmodule
```

A.2 Interface - jk_ff1.sv

```
// Bundle DUT signals for the driver and monitor.
interface jk_ff1;
    logic J;
    logic K;
    logic clk;
    logic rst;
    logic Q;
endinterface
```

A.3 Transaction class

```
class transaction;
    // Randomizable stimulus fields; Q stores the response.
    rand logic J, K, rst;
    logic Q;

    // Print the transaction values for debugging.
    function void display();
        $display("rst=%0b J=%0b K=%0b Q=%0b", rst, J, K, Q);
    endfunction

    // Create a separate object so queued values stay unchanged.
    function transaction copy();
        copy = new();
        copy.J   = this.J;
        copy.K   = this.K;
        copy.rst = this.rst;
        copy.Q   = this.Q;
    endfunction
endclass
```

ANNEXURE A | COMMENTED SYSTEMVERILOG CODE

A.4 Generator class

```
 class generator;
    transaction T1;
    // Typed mailbox: generator sends, driver receives.
    mailbox #(transaction) G2D;
    // Signals that all stimulus has been queued.
    event done;

    // Constructor receives the shared generator-driver mailbox.
    function new(mailbox #(transaction) G2D);
        this.G2D = G2D;
        T1 = new();
    endfunction

    task run();
        // Reset transaction first
        T1.J = 0; T1.K = 0; T1.rst = 1;
        G2D.put(T1.copy());

        // Randomized input combinations after reset
        repeat (12) begin
            if (!T1.randomize())
                $fatal(1, "Transaction randomization failed");
            // Disable reset for the randomized JK tests.
            T1.rst = 0;
            G2D.put(T1.copy());
        end
        // Generation is complete; driving may still be in progress.
        -> done;
    endtask
endclass
```

ANNEXURE A | COMMENTED SYSTEMVERILOG CODE

A.5 Driver class

```
class driver;
    // Handle to the interface instance in the testbench.
    virtual jk_ff1 intf;
    // Typed mailbox: generator sends, driver receives.
    mailbox #(transaction) G2D;
    transaction T1;

    // Constructor receives the shared generator-driver mailbox.
    function new(mailbox #(transaction) G2D);
        this.G2D = G2D;
    endfunction

    task run();
        forever begin
            // Wait until a stimulus transaction is available.
            G2D.get(T1);
            // Drive on the falling edge, before the next DUT update.
            @(negedge intf.clk);
            intf.rst <= T1.rst;
            intf.J   <= T1.J;
            intf.K   <= T1.K;
            $display("DRIVER: Applying inputs");
            T1.display();

        end
    endtask
endclass
```

ANNEXURE A | COMMENTED SYSTEMVERILOG CODE

A.6 Monitor class

```
class monitor;
    // Handle to the interface instance in the testbench.
    virtual jk_ff1 intf;
    // Typed mailbox: monitor sends, scoreboard receives.
    mailbox #(transaction) M2S;
    transaction T1;

    // Constructor receives the shared monitor-scoreboard mailbox.
    function new(mailbox #(transaction) M2S);
        this.M2S = M2S;
    endfunction

    task run();
        forever begin
            // Sample the inputs used by the DUT at this edge.
            @(posedge intf.clk);
            T1 = new();
            T1.J   = intf.J;
            T1.K   = intf.K;
            T1.rst = intf.rst;
            #1; // Let DUT nonblocking assignments update
            T1.Q = intf.Q;
            $display("MONITOR: Observed transaction");
            T1.display();
            // Send an independent observation to the scoreboard.
            M2S.put(T1.copy());
        end
    endtask
endclass
```

ANNEXURE A | COMMENTED SYSTEMVERILOG CODE

A.7 Scoreboard class

```
class scoreboard;
    // Typed mailbox: monitor sends, scoreboard receives.
    mailbox #(transaction) M2S;
    transaction T1;
    // Reference state retained between clock cycles.
    logic expected_Q;
    // Begin checking only after observing a reset.
    bit initialized = 0;

    // Constructor receives the shared monitor-scoreboard mailbox.
    function new(mailbox #(transaction) M2S);
        this.M2S = M2S;
        expected_Q = 1'b0;
    endfunction

    task run();
        forever begin
            // Wait for the next monitored transaction.
            M2S.get(T1);
            // Reset also initializes the reference model.
            if (T1.rst) begin
                expected_Q = 1'b0;
                initialized = 1;
            end
            // Predict the next state using the JK truth table.
            else if (initialized) begin
                case ({T1.J, T1.K})
                    2'b00: expected_Q = expected_Q;
                    2'b01: expected_Q = 1'b0;
                    2'b10: expected_Q = 1'b1;
                    2'b11: expected_Q = ~expected_Q;
                    default: expected_Q = 1'b0;
                endcase
            end
            else begin
                $display("SCOREBOARD: Waiting for reset");
                continue;
            end

              // Case equality gives a definite result even for X/Z values.
              if (T1.Q === expected_Q)
                  $display("PASS: Expected Q=%0b, Actual Q=%0b",
                           expected_Q, T1.Q);

              else
                  $error("FAIL: Expected Q=%0b, Actual Q=%0b",
                         expected_Q, T1.Q);
        end
    endtask
endclass
```

ANNEXURE A | COMMENTED SYSTEMVERILOG CODE

A.8 Top-level testbench - tb.sv

```
`include "jk_ff1.sv"

// Define/include transaction, generator, driver, monitor and scoreboard
// before module tb (or compile their files before this testbench).

module tb;
    // Create the shared signal interface.
    jk_ff1 intf();

    generator gen;
    driver drv;
    monitor mon;
    scoreboard soc;

    // Typed mailbox: generator sends, driver receives.
    mailbox #(transaction) G2D;
    // Typed mailbox: monitor sends, scoreboard receives.
    mailbox #(transaction) M2S;

    // Connect the design ports to the interface signals.
    jk_ff dut (
        .J(intf.J), .K(intf.K), .clk(intf.clk),
        .rst(intf.rst), .Q(intf.Q)
    );

    initial intf.clk = 1'b0;
    // Clock period is 20 simulation time units.
    always #10 intf.clk = ~intf.clk;

    initial begin
        intf.J = 1'b0;
        intf.K = 1'b0;
        intf.rst = 1'b1;
    end

    initial begin
        // Allocate the two shared communication mailboxes.
        G2D = new();
        M2S = new();
        // Construct each component with its corresponding mailbox.
        gen = new(G2D);
        drv = new(G2D);
        mon = new(M2S);
        soc = new(M2S);

          // Connect the driver and monitor to the real interface.
          drv.intf = intf;
          mon.intf = intf;

          // Run generation, driving, monitoring and checking concurrently.
          fork
              gen.run();
              drv.run();
              mon.run();
              soc.run();
          // Allow this initial block to continue without waiting.
          join_none

          // Wait for stimulus generation to finish.
          wait (gen.done.triggered);
          // Allow queued stimulus to be driven and checked.
          repeat (15) @(posedge intf.clk);
          // Allow the final monitor sample and comparison to complete.
          #2;
          $finish;
    end

    initial begin
        // Choose the waveform output file.
        $dumpfile("jk_ff_dump.vcd");
        // Record signals throughout the testbench hierarchy.
        $dumpvars(0, tb);
    end
endmodule
```

EDA Playground: https://edaplayground.com/x/n92Q